1：SGLang 的 RadixAttention/RadixCache 解决什么问题？

SGLang 相当于 LLM 的加速器，那么 RadixAttention 就是加速的核心原理，RadixAttention 解决“重复计算”的问题，它通过 Radix Tree，将所有请求的 KV Cache 按其前缀组织起来，如果新的对话出现重复开头或提示词，就可以直接寻找公共部分，省下重复计算的 token，只用计算后面部分新的 token。

我认为它的优点：能省钱（省 token），省时间。　　缺点：他有缓存，占空间

2：SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分？

Page-sized KV cache 是分页 KV 缓存，解决缓存存放问题，对应显存的分配部分

Prefix reuse 是前缀复用，在寻找相同 KV 缓存前缀的时候，即对应匹配/查前缀部分。

3：vLLM 的 PagedAttention 解决什么问题？它和 SGLang 的 RadixAttention/paged KV 机制有什么联系和差异？

解决空间利用率问题，KV Cache 需要空间放缓存，但这个空间有个问题是利用率不高，PagedAttention 是把 KV Cache 切成一块一块存储，提高空间利用率。就像仓库放上货架可以提高仓库空间利用率。

联系：都是解决缓存空间利用率问题。

差异：SGLang 的 page 可以设的很小，让 RadixCache 做前缀匹配（寻找）更精细而且可以减少内部碎片，代价是计算会略慢。